

ELEC-2320 Design Document

Group 1 | Topic 1 – Grocery Store Staffing Simulation

Matteo Calibani - 110164532

Seth Hansman – 110145578

Rita Petros- 110174358

Derek Robinson – 110071049

Akash Sunilkumar – 110130113

Introduction

A grocery store is reviewing and searching for improvements to its cashier and check-out staffing process. This type of process is important to ensure the successful and efficient operation of the store, as well as the quality of life of customers. With an improvement in this process, staff allocation and the process of checking out will proceed with greater efficiency than before. There are a variety of questions that must be considered when designing the code which will improve these sectors of the grocery store. These questions consist of the following: how will it be decided that the express check-out lanes are staffed, are debit cards available to use on either of the express lanes, what happens if there is a lineup and the staff needs to change shifts, how do customers decide which lane to stand in, and finally, how long does it take cashiers to switch at a till? The answer to these questions will create the foundation for the code that will decide all of these questions and more. In this design document, the written breakdown of the code will be introduced. The reasons for a variety of design decisions will be further explained as well as the process that the code undergoes.

1. Problem Statement and Scope

1.1 Problem Statement:

A grocery store wants a code developed through object-oriented programming which will take different variables or facts about the grocery stores' capabilities and see if new policies fit the needs of the store and customers.

1.2 System's Purpose:

The purpose of the system is to take variables such as the number of items a customer has, whether the customer is paying with cash, cheque, debit or credit, and a small random factor and decide the time taken for the cashier to complete the transaction. Deciding the time taken for a cashier to complete a transaction, different factors considered such as the queue of customers, and the switching of cashiers will allow the grocery store to analyze the data and build new policies to maximize efficiency and quality of life.

2. System Design

2.1 System Overview

The Grocery Store Staffing Simulation is a simulation model for modeling checkout systems, particularly in regard to customers arriving, choosing a line, being served by a cashier, and shift changes for the purpose of evaluating waiting times and utilizing cash registers under various staffing conditions. The simulation processes events such as arrivals and service completions in a specified order without continuous time advancements. The main elements of this simulation model include customers, cashiers, checkout lines, and a simulation controller.

2.2 Class Definitions and Responsibilities

Customer: The Customer class represents an individual customer in the store. It stores the customer's arrival time, number of items, and payment type. It also keeps the service start and finish times so customer waiting time can be calculated.

Cashier: The Cashier class represents a cashier working in the store. It stores the cashier's shift information and whether the cashier is available or busy. It also tracks idle time during the simulation when the cashier is not serving customers.

Checkout Lane: The Checkout Lane class represents a checkout lane in the grocery store. The store contains eight lanes, including regular lanes and express lanes with limits of nine or nineteen items. Each lane holds customers waiting for checkouts and indicates whether a cashier is assigned.

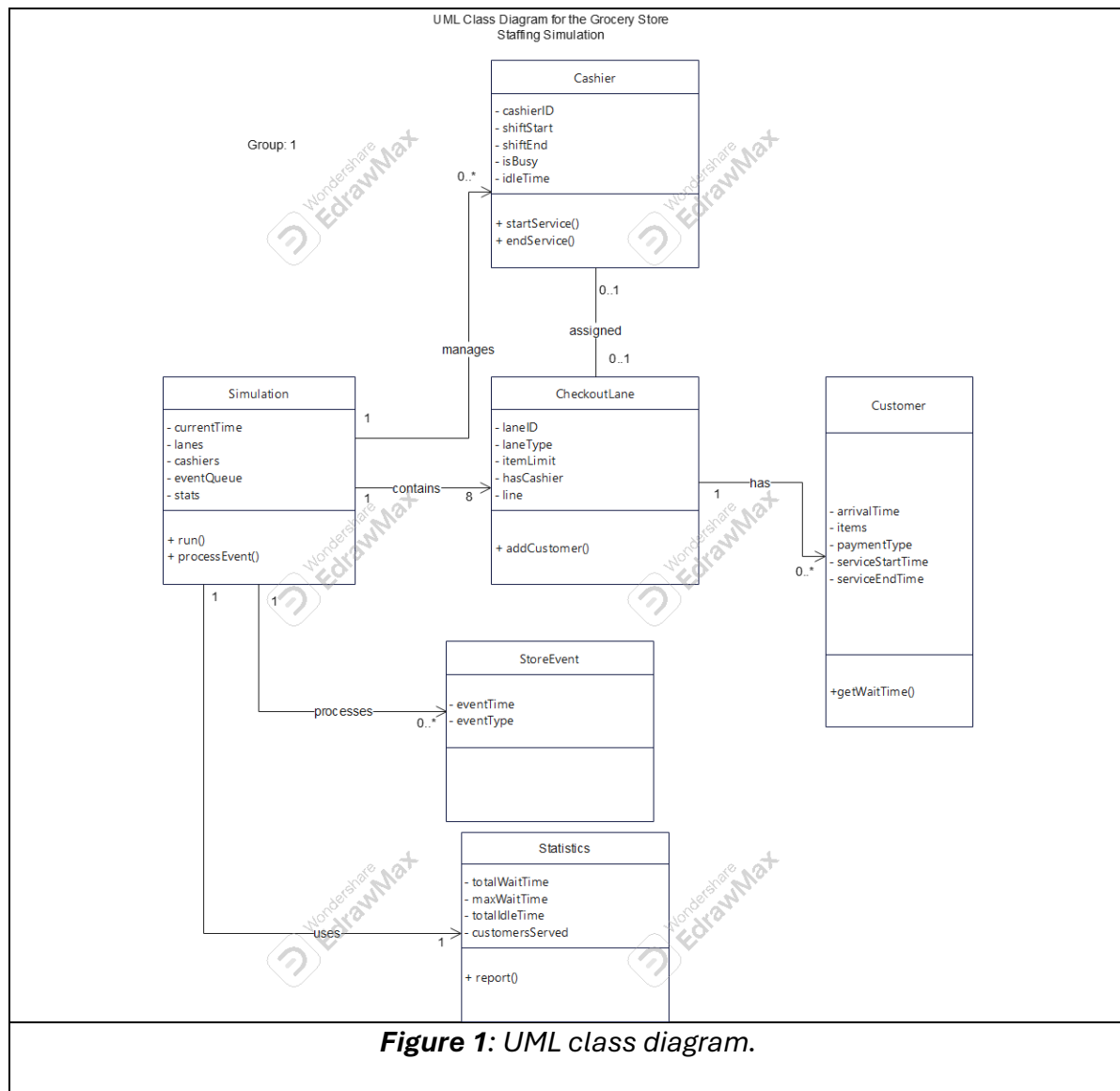
Simulation: The Simulation class is the main part of the program that runs everything. It reads the input files, sets up the lanes and cashiers, and keeps track of the current simulation time. It controls the overall run of the simulation and updates the system as customers arrive, finish service, and as cashier shifts change.

Store Event: The Store Event class represents actions that occur at specific times during the simulation. Examples of events include customer arrivals, service start events, service completion events, and cashier shift changes. Each event stores the time at which it occurs and the type of event

Statistics: The Statistics class is responsible for the simulation results. The results include customer waiting time and cashier idle time. The Statistics class provides a summary of the results at the end of the simulation.

2.3 UML Class Diagram

The diagram below shows the main classes and how they are connected in the system.



2.4 Flowchart

The flowchart below shows how the system operates and how the main parts interact during the simulation.

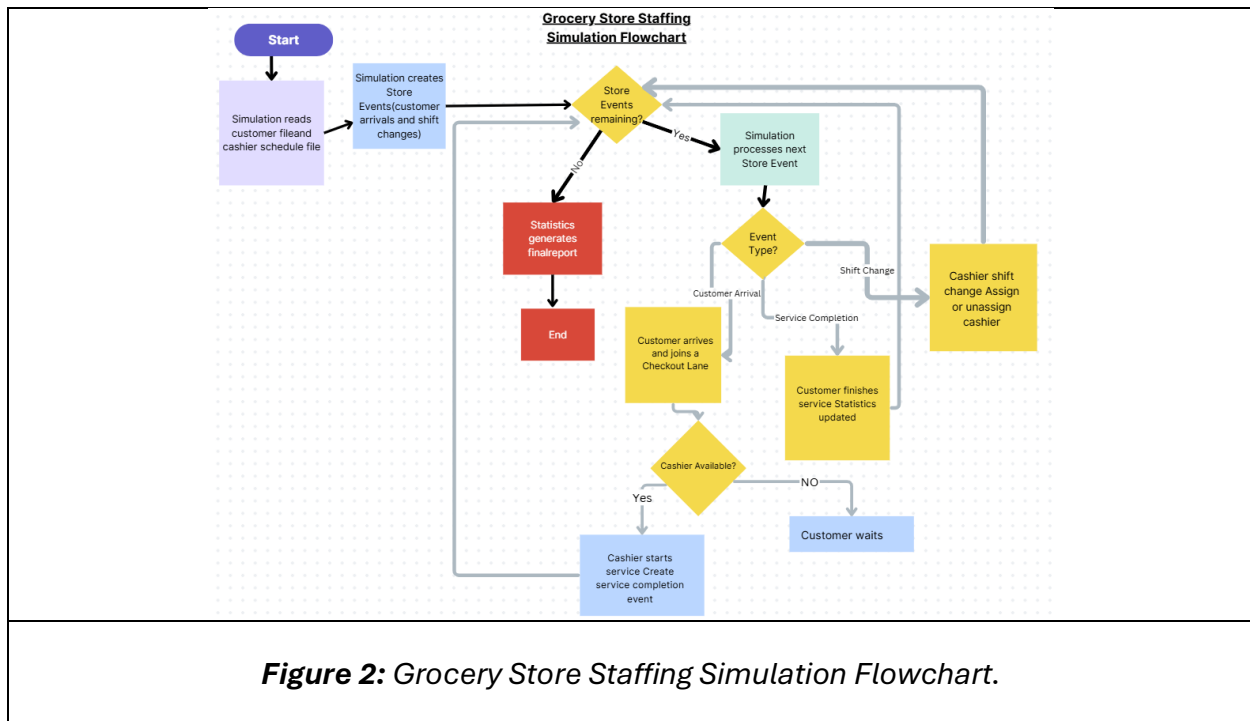


Figure 2: Grocery Store Staffing Simulation Flowchart.

3. Specify Requirements

3.1 Functional Requirements

The Grocery Store Staffing Simulation must be made up of three programs. The first shall take an input from the user and output file containing every customer, their arrival time, number of items, payment type, and possibly the amount of time they will take at checkout to remove calculation time for the third file. The second will output each cashier's shift start, and end times. The third must take the input from the other two programs and output the wait times for each customer, and the idle times for each cashier.

3.2 Non-Functional Requirements

To add to the third program's output for the customers, it will also include the average wait time, maximum wait time, a distribution for the number of customers waiting a specific amount of time, and the wait times over the course of the day. For the cashiers it will also include the average and maximum idle times, and idling times over the course of the day. Each input program should have an ergonomic input for each customer's and cashier's information. Steps shall be taken to reduce computation time and memory use.

4. Describe Algorithms

The Grocery Store Staffing Simulation contains many key logic algorithms in order for proper execution. Important algorithms are used throughout the project to control how time progresses, how customers make decisions, and how service durations are calculated. When working together, these algorithms form the core logic of the simulation.

4.1 Discrete-Event Simulation Algorithm

The discrete-event simulator is the most important algorithm and the brain of the entire simulation. Its job is to represent time, model state changes, and simulate customer interactions.

Algorithm: Simulation Execution

1. Reads customer input and cashier configuration files.
2. Insert all customer arrival events into a priority queue.
3. Insert cashier shift start/end events in priority queue.
4. Initialize simulation clock to 00:00:00.00.
5. Process the event:
 - Arrival: Assign customers to open lanes.
 - Service Completion: Free the cashier and serve the next customer if open.
 - Shift Start: Activate the cashier.
 - Shift End: Deactivate the cashier.
6. Compute and output simulation statistics and overview.

4.2 Customer Lane Selection Algorithm

The customer lane selector dictates which checkout lane they will join after arrival. This decision affects waiting times, service times, and cashier performance.

Algorithm: Lane Selection

1. Identify open lanes.
2. Sort customers into regular lanes or express lanes depending on the item count.
3. Calculate the total remaining service time of customers waiting.
4. Select lane with the smallest remaining service time.
5. Assign customer to the lane.

4.3 Service Time Calculation Algorithm

The service time of each customer depends on the number of items purchased, the method of payment, and a small random factor. These functions work cohesively to create a realistic simulation of cashier transactions.

Algorithm: Service Time Calculation

1. Compute total items processing time.
 - $\text{itemTime} = \text{number_of_items} * 5$
2. Determine payment type and transaction time.
 - 60s Cash
 - 120s Debit
 - 150s Cheque
3. Compute the total service time.
 - $\text{baseTime} = \text{itemTime} + \text{paymentTime}$
4. Apply small random factors to base time.
5. Return the serviceTime.

5. Libraries and Tools

The Grocery Store Staffing Simulation was developed using object-oriented programming principles and several software tools and standard programming libraries to support simulation logic, data management, and program execution.

Programming Language

- **C++** was used as the primary programming language due to its strong support for object-oriented design, efficient memory management, and suitability for event-driven simulations. C++ enables the implementation of classes such as Customer, Cashier, CheckoutLane, StoreEvent, Simulation, and Statistics as described in the system design.

Development Environment

- **Visual Studio / Visual Studio Code** was used as the Integrated Development Environment (IDE) for writing, compiling, debugging, and testing the program. The IDE provides debugging tools that help track simulation events and validate program behavior.

Standard C++ Libraries

The following standard libraries were used to support core functionality:

- `iostream` — Used for console input and output operations to display simulation results and debugging information.
- `vector` — Used to store dynamic collections such as customers, checkout lanes, and event queues.
- `queue` — Used to implement customer waiting lines and event scheduling in the event-driven simulation.
- `string` — Used for handling customer attributes and event descriptions.
- `fstream` — Used for reading input data files such as customer arrivals and cashier schedules.
- `iomanip` — Used to format output results and statistical reports.
- `cstdlib` / `ctime` — Used for generating random values to simulate customer arrivals and transaction variability.

Design and Documentation Tools

- **UML Diagram Tools (e.g., EdrawMax / Draw.io)** were used to create the UML class diagram illustrating system structure and class relationships (shown in Figure 1).
- **Flowchart Design Software (Canva)** was used to model the simulation workflow and event processing logic (shown in Figure 2).
- **Microsoft Word / PDF tools** were used to prepare and format the final design documentation.

These tools and libraries collectively support the implementation of an event-driven simulation capable of modeling customer flow, cashier activity, and performance statistics within the grocery store checkout system.

6. Task Distribution

Readme:

Work done by:
Matteo Calibani

3 Programs to Create in Project

Program 1: Specifying customers (time of arrival at checkouts, # of items, etc.)

Work done by:

Seth Hansman
Rita Petros

Program 2: Specifying Cashiers (time cashier comes on duty, and when cashier comes off duty)

Work done by:

Akash Sunilkumar
Matteo Calibani

Program 3: Runs simulation based on programs 1 and 2

Work done by:

Main – Derek Robinson
Everyone else will help once programs 1 and 2 are completed

Code Debugging/Optimizing:

Work Done by:

Everyone

Conclusion

To conclude this design document, the overall goal of this code is to ensure efficiency and accessibility in terms of reviewing and improving the cashier and till situation at a grocery store. To perform this task, there will be a variety of coding techniques used, such as different classes to call and perform certain tasks, and more described throughout this document. This document specified certain portions of the project such as the breakdown, requirements for the code, certain algorithms, and finally the different libraries and tools used to create this project. Finally, the breakdown of each member's work was specified, which will be followed throughout the duration of the project. In conclusion, all the

information in the design document is how the multiple codes will be put together to create one smooth system for a grocery store.